TPI BASES DE DATOS · UTN SAN RAFAEL

La base de datos como motor de reglas

Bodegas del Atuel

Modelo, integridad y rendimiento en PostgreSQL

Jamardo, Camila (10842)
Sirotiuk, Juliana (10939)

BA

EL RECORRIDO

La idea en una frase

> **La API no calcula nada:**
> **pide operaciones.**
> **PostgreSQL valida, calcula y audita.**

Teoría → código real → evidencia

01 **Modelo y normalización**
DER/MER, 1FN–3FN y jerarquía de clientes.

02 **Integridad y reglas**
Constraints, triggers y procedimientos.

03 **Operación y seguridad**
Vistas, roles y privilegio mínimo.

04 **Rendimiento y demo**
Índices, planes y cuatro evidencias.

ARQUITECTURA

Tres capas, una sola fuente de verdad

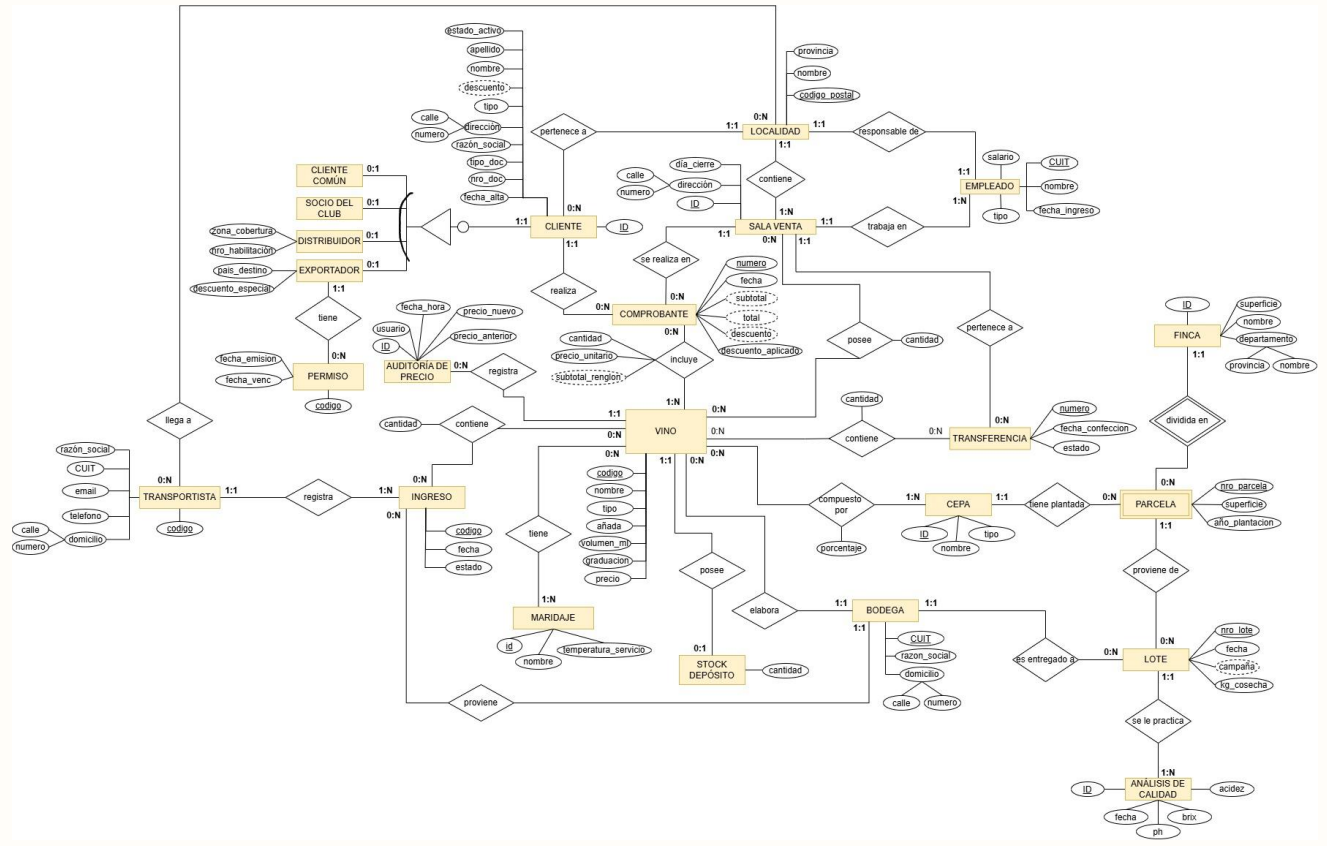


- **El front no decide:** muestra datos y envía operaciones.
- **La API no duplica reglas:** valida formato, arma SQL parametrizado y traduce SQLSTATE a HTTP.
- **La base no es un depósito:** hace cumplir las reglas aunque el cliente sea `psql`, un script o el front.

La regla de negocio no se mudó a la API: sigue viviendo en PostgreSQL.

Del DER al MER

- **Entidades principales:** cliente, vino, comprobante, stock, ingreso y transferencia.
- **1:N con FK:** un cliente tiene muchos comprobantes; una sala tiene muchas ventas.
- **N:M con tabla puente:** `composicion` y `vino_maridaje`.
- **Jerarquía de clientes:** tabla común + subtipos.



DER · 30 tablas · relaciones y jerarquías

DER conceptual

`docs/DER-Bodegas-del-Atuel.jpg`
`docs/MER-Bodegas-del-Atuel.jpg`

1FN · 2FN · 3FN aplicadas

1FN **Atributos atómicos**

Calle y número separados; documento y tipo de documento en columnas propias.

2FN **Dependencia de la clave completa**

En composicion, porcentaje depende de (codigo_vino, id_cepa), no de una parte.

3FN **Sin dependencias transitivas**

Cliente guarda lo común; cada subtipo agrega solo sus atributos.

001_ESQUEMA.SQL

```sql
CREATE TABLE cliente (
  id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  tipo tipo_cliente NOT NULL,
  nombre VARCHAR(60),
  calle VARCHAR(60) NOT NULL,
  numero VARCHAR(10) NOT NULL,
  CONSTRAINT uk_cliente_id_tipo UNIQUE (id, tipo)
);

CREATE TABLE socio_club (
  id_cliente INT PRIMARY KEY,
  tipo tipo_cliente NOT NULL,
  FOREIGN KEY (id_cliente, tipo)
    REFERENCES cliente (id, tipo)
);

CREATE TABLE composicion (
  codigo_vino INT NOT NULL,
  id_cepa INT NOT NULL,
  porcentaje NUMERIC(5,2) NOT NULL,
  PRIMARY KEY (codigo_vino, id_cepa)
);
```

El subtipo no repite nombre, documento ni domicilio: la tabla común guarda lo compartido y cada subtipo solo lo propio.

Las reglas viven en el esquema

002_CONSTRAINTS.SQL

```sql
ALTER TABLE vino
  ADD CONSTRAINT ck_vino_precio_positivo
      CHECK (precio > 0),
  ADD CONSTRAINT uk_vino_nombre_añada_volumen
      UNIQUE (nombre, añada, volumen_ml);

ALTER TABLE renglon_comprobante
  ADD CONSTRAINT ck_renglon_comprobante_cantidad_positiva
      CHECK (cantidad > 0);

ALTER TABLE stock_sala
  ADD CONSTRAINT ck_stock_sala_cantidad_no_negativa
      CHECK (cantidad >= 0);
```

- **CHECK:** precio > 0, cantidades > 0 y stock >= 0.
- **UNIQUE:** no se repite nombre + añada + volumen.
- **FOREIGN KEY:** no hay comprobantes sin cliente ni vinos sin bodega.
- **La API puede validar antes, pero la base es la última palabra.**

RN-01 RN-09 RN-15

Probados en tests/db_tests.sql · 133 verificaciones OK

Lógica reutilizable en la base

003_FUNCIONES.SQL

```sql
CREATE OR REPLACE FUNCTION fn_descuento_cliente(p_id_cliente INT)
RETURNS NUMERIC
LANGUAGE sql STABLE
AS $$
    SELECT COALESCE(
        (SELECT CASE c.tipo
                    WHEN 'cliente_comun' THEN 0.00
                    WHEN 'socio_club' THEN 0.10
                    WHEN 'distribuidor' THEN 0.20
                    WHEN 'exportador' THEN COALESCE(e.descuento_especial, 0.30)
                END
            FROM cliente c
            LEFT JOIN exportador e ON e.id_cliente = c.id
            WHERE c.id = p_id_cliente), 0.00);
$$;
```

- **fn_descuento_cliente:** 0 %, 10 %, 20 % o 30 % según el tipo (RN-06).
- **fn_total_comprobante:** subtotal menos descuento (RN-08).
- **fn_stock_sala:** stock disponible o 0 (RN-09).
- **fn_transicion_valida:** estados válidos (RN-10 y RN-11).
- **Las usan triggers, procedimientos, vistas y tests.**

Se define una vez y se reutiliza: la lógica de negocio no se copia en la API ni en cada consulta.

Una sola puerta de entrada

004_PROCEDIMIENTOS.SQL

```
CREATE PROCEDURE p_procesar_ingreso(p_codigo INT)
...
SELECT estado FROM ingreso
 WHERE codigo = p_codigo
 FOR UPDATE;

IF v_estado <> 'terminado' THEN
  RAISE EXCEPTION 'El ingreso % no esta terminado';
END IF;

INSERT INTO stock_deposito (codigo_vino, cantidad)
SELECT codigo_vino, cantidad
  FROM renglon_ingreso
 WHERE codigo_ingreso = p_codigo
ON CONFLICT (codigo_vino) DO UPDATE
  SET cantidad = stock_deposito.cantidad
               + EXCLUDED.cantidad;
```

- **FOR UPDATE:** bloquea la cabecera.
- **Valida estado:** solo ingreso terminado.
- **Exige renglones:** no procesa un documento vacío.
- **Suma stock con ON CONFLICT.**
- **Estado final:** procesado.

La API solo hace CALL. Evidencia: RN-10 (ingreso) y RN-11 (transferencia).

La venta se calcula sola

- **BEFORE valida; AFTER actualiza.**
- **AFTER INSERT** sobre cada renglón.
- **Misma transacción:** stock y totales.
- **Sin stock suficiente:** excepción y rollback.
- **La API no envía totales:** solo cabecera y renglón.

005_TRIGGERS.SQL

```
CREATE TRIGGER trg_renglon_comprobante_actualiza_venta
AFTER INSERT ON renglon_comprobante
FOR EACH ROW
EXECUTE FUNCTION trg_actualizar_stock_y_totales_venta();

-- dentro de la función
UPDATE stock_sala
   SET cantidad = cantidad - NEW.cantidad
 WHERE id_sala = v_id_sala
   AND codigo_vino = NEW.codigo_vino;

UPDATE comprobante
   SET subtotal = v_subtotal,
       descuento_total = v_descuento_total,
       total = v_total
 WHERE numero = NEW.numero_comprobante;
```

Evidencia: RN-08 calcula totales · RN-09 rechaza la sobreventa y deja el stock intacto.

Una definición, muchos consumidores

006_VISTAS.SQL

```sql
CREATE VIEW vista_stock_total_por_vino AS
SELECT v.codigo, v.nombre,
       COALESCE(d.cantidad, 0) AS stock_deposito,
       COALESCE(s.cantidad, 0) AS stock_sala,
       COALESCE(d.cantidad, 0)
       + COALESCE(s.cantidad, 0) AS stock_total
FROM vino v
LEFT JOIN stock_deposito d
       ON d.codigo_vino = v.codigo
LEFT JOIN (
    SELECT codigo_vino, SUM(cantidad) AS cantidad
    FROM stock_sala
    GROUP BY codigo_vino
) s ON s.codigo_vino = v.codigo;
```

- **Oculta joins y agregaciones.**
- **Métrica consistente:** depósito + salas.
- **LEFT JOIN:** un vino sin stock sigue apareciendo.
- **La usan la ficha del vino y los reportes.**

No es una tabla: se recalcula con los datos actuales y evita duplicar la lógica en cada consulta.

SEGURIDAD

Privilegio mínimo, secretos fuera del repo

007_ROLES.SQL

```
CREATE ROLE rol_api LOGIN;
CREATE ROLE rol_reporte LOGIN;

-- 4. rol_api: lectura y escritura
GRANT SELECT, INSERT, UPDATE, DELETE ON
    bodega, vino, comprobante, stock_sala, ...
TO rol_api;

GRANT SELECT, INSERT ON auditoria_precio TO rol_api;
GRANT EXECUTE ON ALL PROCEDURES
    IN SCHEMA public TO rol_api;

GRANT SELECT ON
    vista_stock_total_por_vino,
    vista_ventas_mensuales_por_sala,
    vista_documentos_pendientes
TO rol_reporte;
```

API

rol_api

DML sobre tablas operativas, auditoría de solo lectura y execute de procedimientos.

REP

rol_reporte

Solo SELECT sobre las tres vistas de reportes.

KEY

Claves por entorno

99_password_api.sql toma APP_DB_PASSWORD y APP_REPORT_PASSWORD. No hay literales.

La API nunca se conecta como owner. /salud devuelve current_user = rol_api.

RENDIMIENTO

Índices: menos buffers, no más código

Consulta	Antes	Después	Mejora
S3	9,753 ms	0,977 ms	10,0×
S8	15,490 ms	9,174 ms	1,69×
S9	77 buf	3 buf	25,7×

Mediana de 3 corridas con EXPLAIN (ANALYZE, BUFFERS).

008_INDICES.SQL

```sql
CREATE INDEX idx_comprobante_sala_fecha
  ON comprobante (id_sala_venta, fecha)
  INCLUDE (id_cliente);

CREATE INDEX idx_comprobante_fecha_sala_total
  ON comprobante (fecha, id_sala_venta)
  INCLUDE (total);
```

No todos los índices ayudan: S1, S11 y S12 siguen con Seq Scan porque leen casi toda la tabla. El índice se justifica con el plan, no con la intuición.

DEMO EN VIVO

Cuatro evidencias en 4 minutos

1. **Venta exitosa** — Totales y stock calculados por la base.
2. **Venta sin stock** — HTTP 409, stock intacto, sin comprobante.
3. **Transferencia** — Crear, terminar y procesar.
4. **Plan de ejecución** — EXPLAIN ANALYZE: Index Only Scan.

QUÉ MIRAR

- HTTP **409** y mensaje de la base.
- Stock **antes = después**.
- **Index Only Scan** y buffers.
- 30 tablas y 796.681 filas.
- **SQLSTATE** traducido a HTTP.

Si la red falla: capturas de respaldo.

`Front  http://localhost:8080  ·  API  http://localhost:8000/docs  ·  Salud  /api/salud`

Si el 8080 está ocupado, el front se publica en 8081 con FRONT_PORT.

Lo que demuestra el TPI

- Modelo normalizado y trazable.
- Reglas ejecutadas por la base, no por la aplicación.
- Rendimiento medido, no supuesto.

Gracias

Jamardo, Camila · 10842
Sirotiuk, Juliana · 10939